# 开源软件基础

## Python 基础

Zhilei Ren

OSCAR Team, School of Software, Dalian University of Technology

November 20, 2017

# Outline

# 为什么选择 Python

- 流行
- 简单 (?)
- 丰富的生态系统
- 交互界面友好

# 所谓 Zen of Python

```
In [1]: import this
The Zen of Python, by Tim Peters
Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!
```

# Python 之禅

Python 之禅 by Tim Peters

- 优美胜于丑陋（Python 以编写优美的代码为目标）
- 明了胜于晦涩（优美的代码应当是明了的，命名规范，风格相似）
- 简洁胜于复杂（优美的代码应当是简洁的，不要有复杂的内部实现）
- 复杂胜于凌乱（如果复杂不可避免，那代码间也不能有难懂的关系，要保持接口简洁）
- 扁平胜于嵌套（优美的代码应当是扁平的，不能有太多的嵌套）
- 间隔胜于紧凑（优美的代码有适当的间隔，不要奢望一行代码解决问题）
- 可读性很重要（优美的代码是可读的）
- 即便假借特例的实用性之名，也不可违背这些规则（这些规则至高无上）
- 不要包容所有错误，除非你确定需要这样做（精准地捕获异常，不写 except:pass）
- 当存在多种可能，不要尝试去猜测
- 而是尽量找一种，最好是唯一一种明显的解决方案（如果不确定，就用穷举法）
- 虽然这并不容易，因为你不是 Python 之父（这里的 Dutch 是指 Guido ）
- 做也许好过不做，但不假思索就动手还不如不做（动手之前要细思量）
- 如果你无法向人描述你的方案，那肯定不是一个好方案；反之亦然（方案测评标准）
- 命名空间是一种绝妙的理念，我们应当多加利用（倡导与号召）
- http://blog.csdn.net/gzlaiyonghao

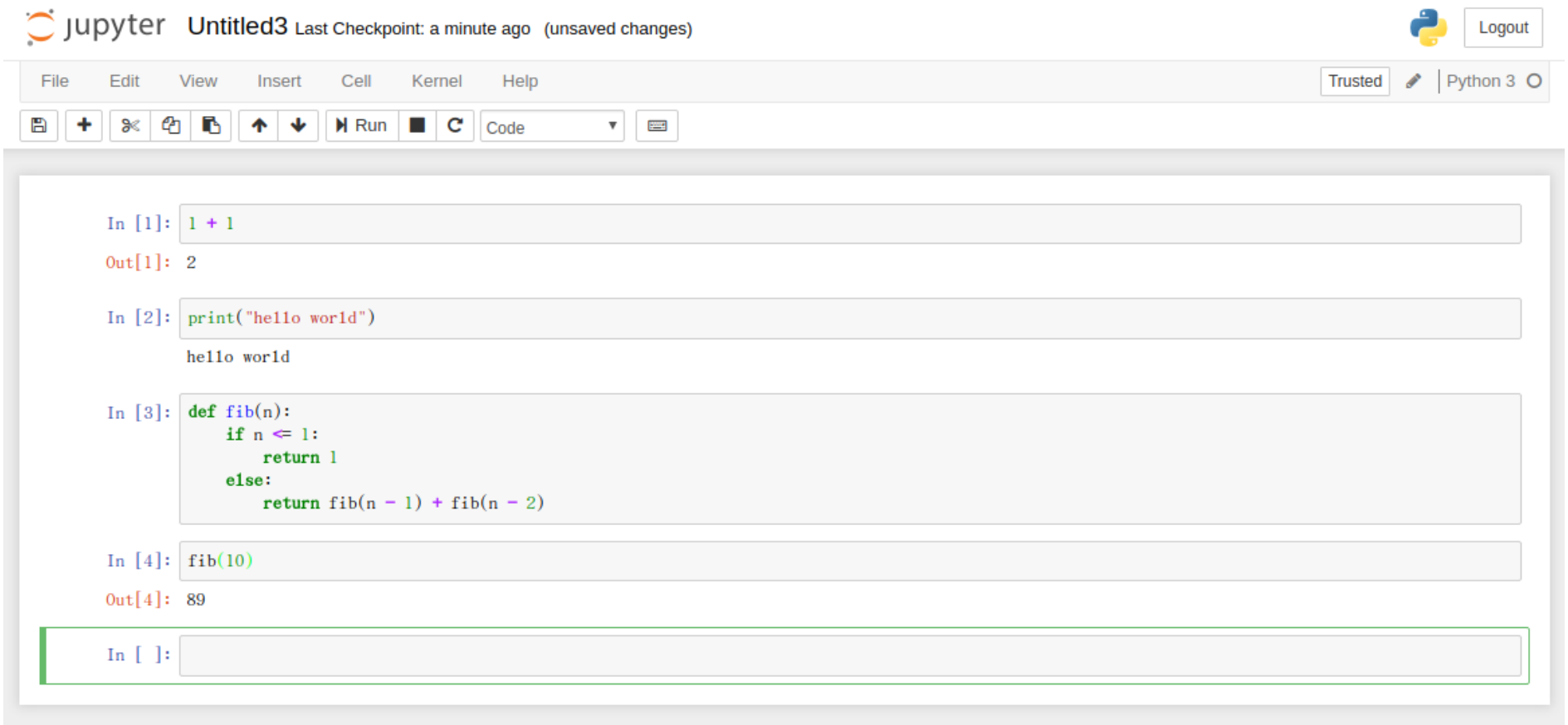
# Python 版本

- Python 存在两个用途广泛的版本：2.x 和 3.x
- 两者不完全兼容
- Python 2.x 分支版本在 2020 年将停止更新
- Python 3 对 UTF-8 支持更好
- 很多公司/开源社区仍默认选择 Python 2

# Python 发行版

- vanilla Python
- Anaconda
- Python (x,y)
- Active Python

# Jupyter Notebook



# 本学期将涉及的 Python 知识

## 知识点

- 流程控制（结构化）
- 面向对象（皮毛）
- 数据获取
- 数据存储
- 数据可视化
- Web 开发（皮毛）

# Outline

# 第一个 Python 程序

```
$ipython3
Python 3.5.2+ (default, Sep 22 2016, 12:18:14)
Type ``copyright'', ``credits'' or ``license'' for more information.

IPython 5.1.0 -- An enhanced Interactive Python.
?         -> Introduction and overview of IPython's features.
%quickref -> Quick reference.
help      -> Python's own help system.
object?   -> Details about 'object', use 'object??' for extra details.

In [1]:
```

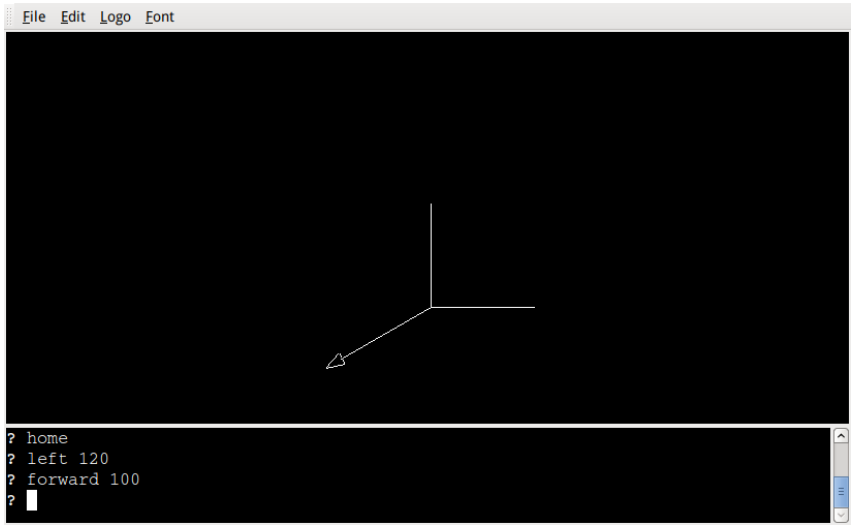
# 第一个 Python 程序

```
In [1]: print("Hello world")
Hello world

In [2]: print(1 + 2)
3

In [3]:
```

# Python Turtle

## Logo

Python Turtle 受 Logo 语言启发而来。

# Python Turtle

```
to tree :len :n
  if :n < 1 [stop]
  forward :len
  left 45
  tree (:len / 2) (:n - 1)
  right 90
  tree (:len / 2) (:n - 1)
  left 45
  back :len
end
```

# Outline

# Outline

# 数值

- Python 解释器可以当计算器使
- 加减乘除都支持
- 括号也能用
- 整形类型为 int
- 有小数点的类型为 float

# 数值

```
>>> 2 + 2
4
>>> 50 - 5*6
20
>>> (50 - 5*6) / 4
5.0
>>> 8 / 5  # 除法返回浮点数
1.6
```

# 数值

- 除法/返回浮点型
- 整数除法用//
- 求模用%
- 幂运算用 **
- 赋值用 =
- 不赋值直接用报错
- 混用整数浮点数类型提升

# 数值

```
>>> 17 / 3  # classic division returns a float
5.666666666666667
>>>
>>> 17 // 3  # floor division discards the fractional
5
>>> 17 % 3  # the % operator returns the remainder of
2
>>> 5 * 3 + 2  # result * divisor + remainder
17
```

# 数值

```
>>> 5 ** 2   # 5 squared
25
>>> 2 ** 7   # 2 to the power of 7
128
```

# 数值

```
>>> width = 20
>>> height = 5 * 9
>>> width * height
900
```

# 数值

```
>>> n  # try to access an undefined variable
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'n' is not defined
```

# 数值

```
>>> 4 * 3.75 - 1
14.0
```

# 数值

- 交互模式下 _ 是特殊变量
- 上一次计算的结果
- 类似 bash 的 $? 或 cmd 的%errorlevel%
- 做计算器很方便
- 非交互模式不能用
- 默认交互环境（非 ipython）下赋值会覆盖
- 除了整数、浮点数，也支持复数

# 数值

```
>>> tax = 12.5 / 100
>>> price = 100.50
>>> price * tax
12.5625
>>> price + _
113.0625
>>> round(_, 2)
113.06
```

# 数值

```
$cat b.py
#!/usr/bin/python3
def foo():
    return 3

foo()
print(_)
```

```
$ python3 b.py
Traceback (most recent call last):
  File ``b.py'', line 7, in <module>
    print(_)
NameError: name '_' is not defined
```

# Outline

# 字符串

- 除了数值，Python 同样支持字符串
- 可以用单引号
- 也可以用双引号
- 没有区别
- \ 用于转义
- 用 r 省去转义（raw）
- """..."""或'''...''' 支持换行
- \ 忽略回车

# 字符串

```
>>> 'spam eggs'  # single quotes
'spam eggs'
>>> 'doesn\'t'  # use \' to escape the single quote...
"doesn't"
>>> "doesn't"  # ...or use double quotes instead
"doesn't"
>>> '"Yes," he said.'
'"Yes," he said.'
>>> "\"Yes,\" he said."
'"Yes," he said.'
>>> '"Isn\'t," she said.'
'"Isn\'t," she said.'
```

# 字符串

```
>>> '"Isn\'t," she said.'
'"Isn\'t," she said.'
>>> print('"Isn\'t," she said.')
"Isn't," she said.
>>> s = 'First line.\nSecond line.'  # \n means newlin
>>> s  # without print(), \n is included in the output
'First line.\nSecond line.'
>>> print(s)  # with print(), \n produces a new line
First line.
Second line.
```

# 字符串

```
>>> print('C:\some\name')   # here \n means newline!
C:\some
ame
>>> print(r'C:\some\name')   # note the r before the qu
C:\some\name
```

# 字符串

```
print("""\
Usage: thingy [OPTIONS]
     -h                        Display this usage mess
     -H hostname               Hostname to connect to
""")
```

# 字符串

- 加号拼接字符串
- 乘号重复字符串
- 字符串常量直接拼接
- 字符串可下标
- 0 起始，-1 最后
- : 用于切片，左闭右开

# 字符串

```
>>> # 3 times 'un', followed by 'ium'
>>> 3 * 'un' + 'ium'
'unununium'
```

# 字符串

```
>>> 'Py' 'thon'
'Python'
```

# 字符串

```
>>> prefix = 'Py'
>>> prefix 'thon'  # can't concatenate a variable and
  ...
SyntaxError: invalid syntax
>>> ('un' * 3) 'ium'
  ...
SyntaxError: invalid syntax
```

# 字符串

```
>>> prefix + 'thon'
'Python'
```

# 字符串

```
>>> text = ('Put several strings within parentheses '
...         'to have them joined together.')
>>> text
'Put several strings within parentheses to have them j
```

# 字符串

```
>>> word = 'Python'
>>> word[0]  # character in position 0
'P'
>>> word[5]  # character in position 5
'n'
```

# 字符串

```
>>> word[-1]   # last character
'n'
>>> word[-2]   # second-last character
'o'
>>> word[-6]
'P'
```

# 字符串

```
>>> word[0:2]  # characters from position 0 (included)
'Py'
>>> word[2:5]  # characters from position 2 (included)
'tho'
```

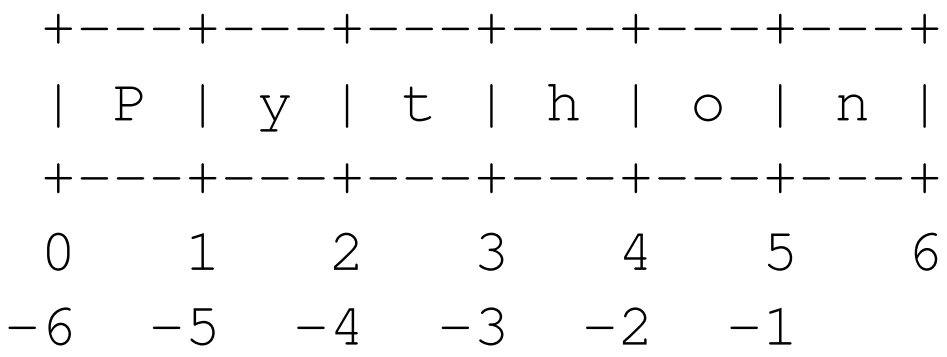
# 字符串

```
>>> word[:2] + word[2:]
'Python'
>>> word[:4] + word[4:]
'Python'
```

# 字符串

```
>>> word[:2]    # character from the beginning to posit
'Py'
>>> word[4:]    # characters from position 4 (included)
'on'
>>> word[-2:]   # characters from the second-last (incl
'on'
```

# 字符串



# 字符串

```
>>> word[42]  # the word only has 6 characters
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: string index out of range
```

# 字符串

```
>>> word[4:42]
'on'
>>> word[42:]
''
```

# 字符串

- Python 字符串是不可变的，与 Java 类似
- len 返回长度，类比 strlen

# 字符串

```
>>> word[0] = 'J'
  ...
TypeError: 'str' object does not support item assignme
>>> word[2:] = 'py'
  ...
TypeError: 'str' object does not support item assignme
```

# 字符串

```
>>> 'J' + word[1:]
'Jython'
>>> word[:2] + 'py'
'Pypy'
```

# 字符串

```
>>> s = 'supercalifragilisticexpialidocious'
>>> len(s)
34
```

# Outline

# 列表

- Python 支持若干数据类型
- 最常见的是列表
- 与数组类似
- 允许元素类型不同
- 和字符串类似，可下标可切片
- 加法和乘法也和字符串类似
- len 返回长度
- 与字符串不同的，列表是可变的
- append 方法用于追加
- 多维数组用嵌套列表实现

# 列表

```
>>> squares = [1, 4, 9, 16, 25]
>>> squares
[1, 4, 9, 16, 25]
```

# 列表

```
>>> squares[0]  # indexing returns the item
1
>>> squares[-1]
25
>>> squares[-3:]  # slicing returns a new list
[9, 16, 25]
```

# 列表

```
>>> squares[:]
[1, 4, 9, 16, 25]
```

# 列表

```
>>> squares + [36, 49, 64, 81, 100]
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

# 列表

```
>>> cubes = [1, 8, 27, 65, 125]  # something's wrong h
>>> 4 ** 3  # the cube of 4 is 64, not 65!
64
>>> cubes[3] = 64  # replace the wrong value
>>> cubes
[1, 8, 27, 64, 125]
```

# 列表

```
>>> cubes.append(216)  # add the cube of 6
>>> cubes.append(7 ** 3)  # and the cube of 7
>>> cubes
[1, 8, 27, 64, 125, 216, 343]
```

# 列表

```
>>> letters = ['a', 'b', 'c', 'd']
>>> len(letters)
4
```

# 列表

```
>>> a = ['a', 'b', 'c']
>>> n = [1, 2, 3]
>>> x = [a, n]
>>> x
[['a', 'b', 'c'], [1, 2, 3]]
>>> x[0]
['a', 'b', 'c']
>>> x[0][1]
'b'
```

# Outline

# if Statements

- if 用于分支
- True/False 表示真假
- None/False/0/”/()/[]/{} 会判定为假
- 可以有任意多个 elif 分支
- 缩进代表嵌套深度

# if Statements

```
>>> x = int(input("Please enter an integer: "))
Please enter an integer: 42
>>> if x < 0:
...     x = 0
...     print('Negative changed to zero')
... elif x == 0:
...     print('Zero')
... elif x == 1:
...     print('Single')
... else:
...     print('More')
...
More
```

# for Statements

- for 用于循环
- 可以在列表、字符串上遍历
- 可以用 range 生成遍历序列
- range 左闭右开
- range 在 Python3 中不返回列表，可用 list(range()) 习语
- 缩进代表嵌套深度

# for Statements

```
>>> # Measure some strings:
... words = ['cat', 'window', 'defenestrate']
>>> for w in words:
...     print(w, len(w))
...
cat 3
window 6
defenestrate 12
```

# The range Function

```
>>> for i in range(5):
...     print(i)
...
0
1
2
3
4
```

# Outline

# 函数定义

- def 开头，代表定义函数
- def 和函数名中间要敲一个空格
- 之后是函数名，这个名字用户自己起的，方便自己使用就好
- 函数名后跟圆括号 ()，代表定义的是函数，里边可加参数
- 圆括号 () 后一定要加冒号: 这个很重要，不要忘记了
- 代码块部分，是由语句组成，要有缩进
- 函数要有返回值 return
- 调用时使用函数名 (参数) 方式

# 函数

```
def foo():
    print("hello world")

def add(x, y):
    return x + y

print(add(3, 4))
```

# 函数

```
import turtle
def square(length):
    turtle.forward(length)
    turtle.left(90)
    turtle.forward(length)
    turtle.left(90)
    turtle.forward(length)
    turtle.left(90)
    turtle.forward(length)
    turtle.left(90)
```

# 函数

左转，左转，左转，左转，好像是个圈哟

# 函数

```
def square(length):
    for i in range(4):
        turtle.forward(length)
        turtle.left(90)
```

# 函数

```
def square(length):
    for i in range(4):
        turtle.forward(length)
        turtle.left(90)
    square(length + 10)
```

# Outline

# 递归

To understand recursion, first you have to understand recursion.

# 阶乘

```
def factorial(n):
    if n <= 1:
        return 1
    else:
        return n * factorial(n - 1)
```

# 递归

With a little help from my friends.

– The Beatles

# 分形

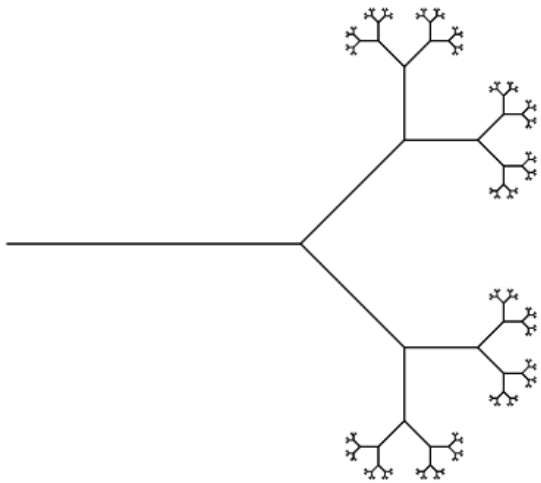
分形（Fractal），又称碎形、残形，通常被定义为「一个粗糙或零碎的几何形狀，可以分成数個部分，且每一部分都（至少近似地）是整体缩小后的形狀」，即具有自相似的性质。[1]

[1] https://zh.wikipedia.org/wiki/

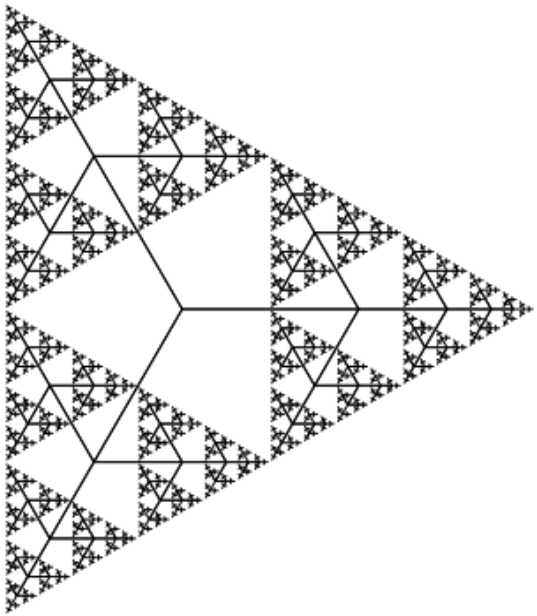
# Turtle

```
import turtle

def tree(length,n):
    """ paints a branch of a tree """
    if length < (length/n):
        return                          # 边界条件
    turtle.forward(length)              # 主干
    turtle.left(45)                     # 左转分叉
    tree(length * 0.5,length/n)         # 左子树
    turtle.right(90)                    # 右转分叉
    tree(length * 0.5,length/n)         # 右子树
    turtle.left(45)                     # 转回
    turtle.backward(length)             # 回到起点
    return                              # 返回
```

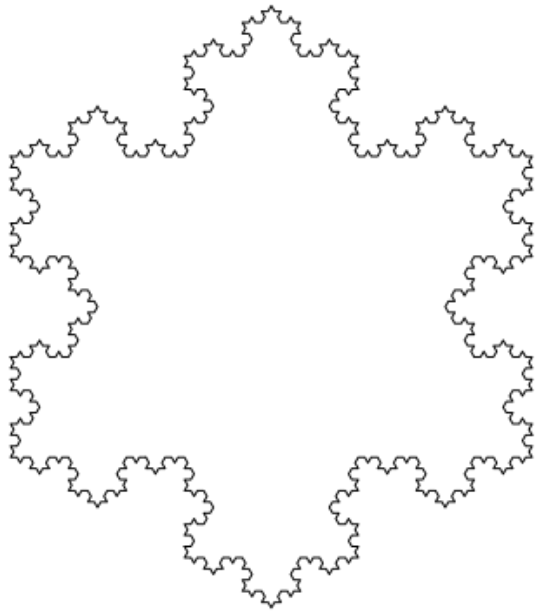
# Turtle



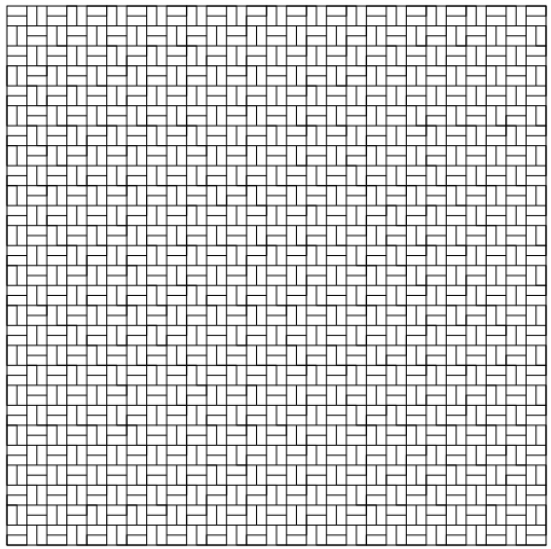
# Turtle



# Turtle



# Turtle



# Outline

# 面向对象

## 面向对象三要素

- 继承：儿子点出父亲成员
- 封装：点号能点出成员
- 多态：儿子点出和父亲不同的行为

# 类的声明

```
class ClassName:
    <statement-1>
    .
    .
    .
    <statement-N>
```

# Class Objects

```
class MyClass:
    """A simple example class"""
    i = 12345

    def f(self):
        return 'hello world'
```

# 类的成员

# 成员可见性

## 成员可见性

- 不存在类似 C++ 的私有
- 约定：下划线前缀的是实现细节
- ipython/Jupyter 默认不会补全
- 类似 *nix 下的 dotfiles

# __init__ 函数

```
def __init__(self):
    self.data = []
```

# Class Objects

```
>>> class Complex:
...     def __init__(self, realpart, imagpart):
...         self.r = realpart
...         self.i = imagpart
...
>>> x = Complex(3.0, -4.5)
>>> x.r, x.i
(3.0, -4.5)
```

# 成员变量和实例变量

```
class Dog:

    tricks = []                 # mistaken use of a class

    def __init__(self, name):
        self.name = name

    def add_trick(self, trick):
        self.tricks.append(trick)

>>> d = Dog('Fido')
>>> e = Dog('Buddy')
>>> d.add_trick('roll over')
```

# 鸭子类型

在程序设计中，鸭子类型（英语：duck typing）是动态类型的一种风格。在这种风格中，一个对象有效的语义，不是由继承自特定的类或实现特定的接口，而是由" 当前方法和属性的集合" 决定。这个概念的名字来源于由 James Whitcomb Riley 提出的鸭子测试，“鸭子测试”可以这样表述：

## 鸭子测试

- If it walks like a duck
- It quacks like a duck
- Then it must be a duck

“当看到一只鸟走起来像鸭子、游泳起来像鸭子、叫起来也像鸭子，那么这只鸟就可以被称为鸭子。”[2]

[2] https://en.wikipedia.org/wiki/Duck_typing

# 继承

# 多态

# Outline

# 函数一等公民

# λ 演算

# List Comprehension

# Lisp

# Haskell

# 生成器

# Map/Filter/Reduce